Project Documentation:
Web Technologies 2025/26 Group Project

Alasotto Cristian, Collura Federico, Correndo Davide

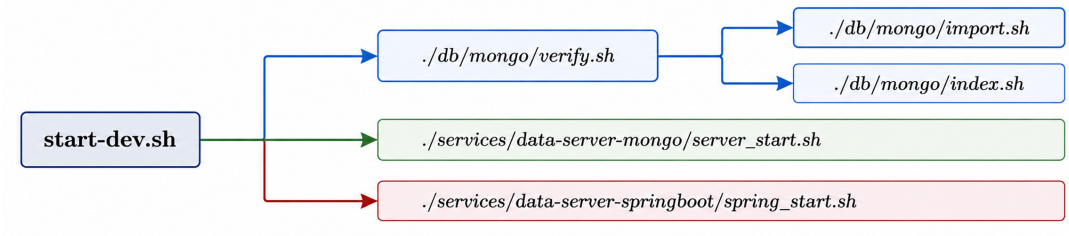
1 Introduction

This project implements a web platform for exploring and analysing anime-related data. Users can browse anime, characters and authors, apply filters and sorting, and submit ratings through an intuitive web interface.

The system is built using a multi-server architecture with a central Express server acting as a lightweight middleware between the client and two data services: a Spring Boot service with a PostgreSQL database for static data, and an Express service with MongoDB for dynamic data. The front-end is developed with HTML, CSS, JavaScript and Handlebars templates.

The project follows the design principles and technologies introduced in the course, focusing on modularity, clear separation of concerns and well-documented APIs.

The project startup is fully automated through Docker containers and can be executed with a single command. To run the system, ensure that Docker and Node.js are installed and that the dataset files are placed in the `solution/data` folder. For the simplest setup, navigate to the `solution` directory and run `./start-dev.sh`; this script starts the databases, imports the PostgreSQL data and launches all services.



Alternatively, you can start the containers manually using `docker compose up`, install dependencies in the `main-server-express` directory and run `npm run dev`. MongoDB data are imported, indexed and verified through the scripts in `./db/mongo`. The system uses environment variables defined in `.env` to configure service URLs and logging levels. Once running, the web interface is accessible at `http://localhost:3000`.

2 Technical Task 1 – System Architecture and Data Management

2.1 Design and Motivations

The system follows a microservice architecture with a central Express server acting as the entry point and coordinating access to two data services. Static data are stored in PostgreSQL and served by a Spring Boot application, while dynamic data are stored in MongoDB to support high write volumes.

The central server communicates with both services via Axios, ensuring a clear separation of concerns and keeping the middleware lightweight. The front-end is rendered server-side using Handlebars, as required by the assignment. Docker Compose orchestrates all services, with configuration managed through environment variables and a start-up script to simplify development and data loading.

2.2 Issues

Major challenges included handling large datasets, indexing MongoDB collections for performance, mapping a complex relational schema in PostgreSQL, and coordinating asynchronous calls between services. Additional effort was required to validate user input and correctly configure Docker networking across environments.

2.3 Requirements

The architecture meets the assignment requirements by separating static and dynamic data, using Express and Spring Boot, and keeping the central server lightweight. All services expose REST APIs with pagination, filtering and sorting, are fully containerised with Docker Compose, and provide Swagger documentation.

3 Technical Task 2 – Postgres Data Service (Spring Boot)

3.1 Structure and Functioning

The Spring Boot service is organised into ten modules, each covering one database entity, all following a three-tier structure: `Controller` (HTTP requests), `Service` (business logic, DTO), and `Repository` (PostgreSQL).

Each module uses a DTO to decouple the JPA entity from JSON. The DTO is built via `fromEntity()`, exposes only getters, and is read-only. Entities never leave the service layer: all methods return a DTO or `Page<DTO>`. Field names use snake_case via `@JsonProperty`.

The service layer includes `likePattern()`, which builds `%value%` for text search, ensuring Hibernate receives a non-null `String` and infers `VARCHAR`. The database has ten tables with 28,955 anime records; six use composite keys with `@IdClass` implementing `Serializable`, `equals()`, and `hashCode()`. Junction tables (e.g. `character_anime_works`, `person_voice_works`) support JPQL cross-entity queries.

All endpoints support page-based and limit/offset pagination (default ten results) and a `sort` parameter (comma-separated, - for descending); NULL values are last. Swagger documents endpoints at `http://localhost:8080/swagger-ui/index.html`, and Javadoc documents all classes and methods.

3.2 Development History and Problems

Development had three phases: initial structure with NULL filtering and Swagger, bug fixing, and replacing inline comments with Javadoc.

Bug 1 – Combined filters ignored. `findWithFilters()` in `DetailsService` used `if/return` chains, so once one filter was non-null, others were ignored.
A request like `?type=TV&rating=PG-13&genres=Action` returned all 1,683 TV anime instead of 50–100 matching all filters, with results depending on parameter order. The fix introduced `findWithCombinedFil` a JPQL query applying all filters with AND logic using `:param IS NULL OR condition`.

Bug 2 – PostgreSQL casting error. Text search failed with `function lower(bytea) does not exist` due to `LOWER(CAST(e.title AS string))`, translated to `varchar` but interpreted as `bytea`.

Attempts like `AS text` failed. The fix removed `CAST()` and used `LOWER(e.title)`, since the column is already `VARCHAR`.

Refactoring of a previous approach. A JPA Specifications approach for multi-filtering was abandoned due to complexity and poor pagination integration, and replaced by the JPQL combined query.

3.3 Requirements

The service uses JPA with `@Query` JPQL only (no native SQL), ensuring camelCase to snake_case mapping. Configuration uses environment variables via Docker Compose. Cross-entity endpoints (e.g. `/api/details/{mal_id}/characters`) use JPQL subqueries through junction tables.

3.4 Limitations

Text search uses case-insensitive `LIKE` only (no full-text search). Cross-entity JPQL subqueries may be slower than SQL JOINs on large datasets. Only one endpoint supports writes (`POST /api/details/update_score`); all others are read-only.

4 Technical Task 3 – MongoDB Data Service (Node.js)

4.1 Design and Motivations

The MongoDB server is a Node.js application using Express and the Mongoose Object Data Modeling (ODM) library. It manages the three dynamic collections: `ratings`, `favs`, and `stats`.

The `/api/{collection_name}` endpoints allow clients to retrieve ratings by anime identifier, apply filters, and paginate results. Clients can also submit ratings via a POST request; the API writes the new rating and updates the corresponding aggregate statistic, such as average score and vote counts.

All endpoints return data with optional filters. To streamline the architecture, we implemented a direct Model-Controller pattern: controllers handle HTTP requests and interact directly with the Mongoose models, meeting the specific architectural constraints of the Model-Controller pattern.

4.2 Issues

The primary challenges in this server were related to data volume and consistency. The `ratings` collection alone contains hundreds of millions of documents, so queries must use indexes to avoid full collection scans. Creating these indexes during deployment adds several minutes to the setup process.

When users submit new ratings, we need to validate the request and check that the username exists in the PostgreSQL database; the server delegates this check to the central server's API client, introducing latency.

Updating aggregate statistics in MongoDB requires careful handling to avoid race conditions; we chose to use an atomic `findOneAndUpdate` operation with `$inc` modifiers to securely update raw counts, followed by dynamic calculation of weighted averages and score percentage breakdowns.

Ensuring that our API adheres to REST principles while also providing flexibility for advanced queries required designing a consistent parameter scheme, for example `minScore` and `maxScore` for numeric filters.

4.3 Requirements

This server fulfils the requirement to store dynamic data in MongoDB and to implement at least one server in Node.js. It exposes REST endpoints for retrieving and creating ratings, favourites, and statistics, uses pagination, and supports advanced filtering options.

The server is documented with Swagger and JSDoc: the Swagger documentation is served at `http://localhost:3001/api-docs`, while the JSDoc documentation is available in a documentation folder named `jsdoc`.

Currently, the MongoDB connection string and database configuration are statically defined within the codebase, using `localhost:27017`. Data import scripts populate the collections and create indexes; the README explains how to run these scripts. Proper status codes and error messages are returned for missing data or invalid requests.

Data are also cached to improve performance by avoiding the execution of the same functions for different users when the underlying data have not changed.

4.4 Limitations

Current queries on the `ratings` collection rely on simple comparison operators and do not support more complex aggregations such as time-series analysis or grouping by user. The POST endpoint for ratings does not yet support updating existing ratings; repeated submissions create duplicate documents.

The server trusts the central server's validation and does not implement its own user authentication. While the design includes indexes on frequently queried fields, the large size of the collections can result in non-trivial database processing times when retrieving non-cached data on commodity hardware. However, the server uses asynchronous Promises to fetch ratings, making requests non-blocking and ensuring that users do not have to wait several seconds for the page to load.

5 Technical Task 4 – Central Express Server and Front-end

5.1 Central Server – Design and Motivations

The central server is implemented in Node.js using Express and Handlebars and acts as the main entry point for user interaction. It serves as a lightweight middleware between the client and two back-end data services (Spring Boot/PostgreSQL and Express/MongoDB), complying with the assignment requirement to avoid heavy computation. Communication with the data services is handled via Axios clients, and routes such as `/anime`, `/anime/:id`, `/anime/:id/ratings` and `/profile/:username` translate user requests into appropriate API calls. Handlebars templates with partials and helpers are used for server-side rendering of lists, filters and pagination while preserving query parameters across navigation. This approach simplifies the front-end stack and ensures full compliance with the assignment constraints. Compared to a React-based solution, server-side rendering with Handlebars reduces client-side complexity and avoids the need for a separate front-end build process, but it limits interactivity and prevents dynamic UI updates without page reloads.
A caching mechanism is implemented in the MongoDB data server, because that server directly handles dynamic and high-volume collections such as ratings, favourites and statistics. Caching there is useful to reduce repeated database accesses and improve response time for frequently requested dynamic data. In contrast, the main-server-express does not implement its own caching layer, because its role is not to access the databases directly, but to orchestrate requests, render Handlebars pages, and forward calls to the data servers through Axios. Adding another cache in the main server would increase complexity and could introduce consistency problems, especially for dynamic data that may change frequently.

5.2 Issues

Key challenges included mapping user input into valid API queries while preserving active filters across pagination. Input validation and normalisation were required before forwarding requests to the data services. Some pages require data aggregation from both PostgreSQL and MongoDB, which involved coordinating multiple asynchronous calls and handling partial service failures. Designing a clear interface using only HTML and Handlebars required careful layout planning.

5.3 Requirements

The central server satisfies the assignment requirements by using Express as the main gateway and Handlebars for templating. It supports filtering, sorting and pagination aligned with the underlying APIs. Configuration is managed via environment variables, including service URLs and logging options. Rating submission is handled through POST forms, with username validation delegated to the PostgreSQL service before forwarding data to MongoDB. The server is documented with Swagger and JSDoc: the Swagger documentation is served at `http://localhost:3000/api-docs`, while the JSDoc documentation is available at `http://localhost:3000/jsdoc`.

5.4 Limitations

The application does not implement authentication or advanced personalisation features. UI interactions are page-based and lack dynamic behaviours such as live filtering or auto-complete, which a React front-end could provide. While the chosen architecture ensures simplicity and robustness, a React-based client could offer a more responsive user experience at the cost of increased architectural and implementation complexity.

6 Conclusions

Building a multi-service web application that integrates relational and document-oriented data taught us valuable lessons about system design, data modelling and cross-technology integration. We realised early on that clear separation of concerns and consistent API design make it easier to manage complexity. Containerisation with Docker simplified deployment and allowed us to replicate the environment across different development machines. Handling large datasets required tuning indexes and writing efficient queries; we learned to measure performance. Collaboration across different programming languages fostered better teamwork, as each member had to understand how their component interacted with the others. While our implementation meets the essential requirements, numerous opportunities remain for improvement, such as adding authentication with password managment and caching.

7 Division of Work

We divided tasks to leverage each member's strengths while ensuring that everyone contributed to all parts of the project.
Member Alasotto Cristian focused on the central Express server and front-end, designing the Handlebars templates, implementing route logic, ensuring that query parameters were correctly mapped to API calls, and creating and managing the Docker containers used to run the project services.
Member Collura Federico took primary responsibility for the Spring Boot service, modelling the PostgreSQL schema in Java, writing custom queries and integrating Swagger documentation.
Member Correndo Davide led the development of the MongoDB service, writing controllers and services for ratings, favourites and statistics, and implementing data import and indexing scripts.